

# OPSEC & Detection-Aware Operations

Operational Risk, Telemetry, Detection Engineering, Cleanup, and Safe Validation

### Prerequisites

- Module 00-23, terutama evidence handling, Windows/Linux fundamentals, ATT&CK, adversary emulation, dan C2 telemetry.
- Omarchy/Arch Linux dengan Python 3, jq, dan systemd journal; Windows VM + Sysmon/Windows Security auditing bila tersedia.
- Semua test dilakukan hanya pada host sendiri, disposable VM, isolated lab, atau platform training yang secara eksplisit mengizinkan pengujian.

### Tooling di Omarchy

omarchy pkg add python jq ripgrep tcpdump wireshark-qt

Optional untuk lab yang lebih lanjut: audit userspace tools di Linux, Sysmon di Windows, Sigma tooling, dan Atomic Red Team pada disposable test machine. Jangan memasang tool dari sumber acak hanya karena namanya populer.

**Batas module** - Detection-aware operation bukan panduan bypass AV/EDR. Module ini tidak mengajarkan mematikan security controls, log tampering, AMSI/ETW bypass, unhooking, payload obfuscation, process injection, timestomping, atau persistence tersembunyi. Kita belajar memahami footprint dan meningkatkan kualitas operasi + deteksi.

## 1. OPSEC dalam Red Team: Mengelola Risiko Operasional

Operational Security (OPSEC) pada engagement yang authorized berarti mengendalikan risiko agar pengujian mencapai objective tanpa menyebabkan outage, data exposure, scope violation, atau meninggalkan artefak yang tidak diperlukan. “Tidak terdeteksi” bukan objective universal. Kadang engagement justru bertujuan menguji apakah control mendeteksi behavior tertentu. OPSEC yang matang menjawab: apa objective, apa yang boleh berubah, apa yang tidak boleh disentuh, siapa yang menanggung risiko, bukti apa yang dikumpulkan, dan bagaimana keadaan dikembalikan setelah test.

### 1.1 OPSEC decision model

| Pertanyaan      | Contoh jawaban production-minded                                                            |
|-----------------|---------------------------------------------------------------------------------------------|
| Objective       | Validasi bahwa discovery chain menghasilkan telemetry yang bisa dikorelasikan.              |
| Scope           | VM training-01 dan SIEM lab; tidak mencakup workstation pengguna.                           |
| Allowed change  | Membuat file marker di folder lab; tidak boleh mengubah policy/security control.            |
| Exposure budget | Satu host, satu user training, <= 20 request, <= 10 menit.                                  |
| Stop condition  | Service error, CPU abnormal, unexpected data, atau alert severity tinggi di luar hipotesis. |
| Evidence        | UTC timestamps, event IDs, hashes, pcap/log snippet, cleanup proof.                         |
| Cleanup owner   | Operator yang membuat artefak wajib menghapus dan memverifikasi residual.                   |

### 1.2 Artifact lifecycle

PLAN -> STAGE -> EXECUTE -> OBSERVE -> COLLECT -> CLEAN -> VERIFY -> ARCHIVE

Setiap tool, script, temporary file, listener, test account, scheduled execution, certificate, firewall rule, container, atau credential lab harus memiliki owner dan cleanup state. Kalau operator tidak tahu apa yang dibuat selama test, cleanup tidak bisa dianggap selesai.

### 1.3 OPSEC budget bukan “stealth score”

Gunakan budget untuk mengendalikan blast radius: jumlah host, durasi, request rate, privilege, data class, perubahan state, dan artefak. Tujuannya adalah predictable testing. Hindari anti-pattern “semakin tersembunyi semakin profesional”; perilaku yang sengaja dibuat tidak terlihat dapat merusak validitas detection test dan meningkatkan risiko engagement.

## 2. Footprint: Apa yang Dilihat Defender

Satu aksi operator dapat meninggalkan banyak jejak. Detection-aware operator memikirkan footprint sebelum mengeksekusi: process tree, command line, file write, registry/config change, DNS, network flow, authentication, service/task activity, memory behavior, cloud audit, dan application logs. Kualitas operasi naik ketika kita dapat memprediksi jejak ini dan membandingkannya dengan telemetry yang benar-benar tersedia.

| Footprint       | Pertanyaan red-team                          | Contoh telemetry defender                        |
|-----------------|----------------------------------------------|--------------------------------------------------|
| Process         | Program apa, parent apa, user/token apa?     | Sysmon EID 1; Windows 4688; process audit/EDR.   |
| Network         | Destination, protocol, timing, bytes?        | Sysmon EID 3; firewall/NetFlow/proxy/pcap.       |
| DNS             | Query siapa, domain apa, frekuensi?          | Sysmon EID 22; resolver logs; DNS telemetry.     |
| File            | Path, hash, create/modify/delete?            | Sysmon EID 11; EDR file events; audit logs.      |
| Registry/config | Apa yang berubah dan siapa yang mengubah?    | Sysmon registry events; config-management/audit. |
| Auth            | Account, source, target, logon type, result? | Windows Security; IdP/SSO/VPN logs.              |
| Service/task    | Install/start/change?                        | SCM/Task Scheduler/Sysmon/Security events.       |
| Cloud/API       | Actor, resource, action, source, token?      | Provider audit logs / identity telemetry.        |

### 2.1 Command line adalah telemetry sekaligus risiko data

Windows Event 4688 dapat mencatat process creation; command line hanya tersedia jika policy “Include command line in process creation events” diaktifkan. Microsoft juga memperingatkan bahwa command-line arguments dapat mengandung password atau data sensitif. Implikasi untuk operator: jangan taruh secret di CLI kalau ada alternatif stdin, environment-scoped secret store, file descriptor, atau prompt interaktif.

### 2.2 Linux observability

Pada Linux, journalctl membaca structured systemd journal. Auditd adalah userspace component Linux Auditing System; ausearch/aureport digunakan untuk membaca audit records jika audit subsystem tersedia. Untuk lab, kita mulai dari read-only observation: journalctl, ps, ss,

systemctl, dan optional audit queries. Jangan menambahkan audit rules pada production host tanpa approval karena dapat memengaruhi volume dan operasi logging.

2.3 Telemetry planes dan blind spot

| Plane       | Contoh                               | Blind spot umum                                                          |
|-------------|--------------------------------------|--------------------------------------------------------------------------|
| Endpoint    | Process, file, registry, module, DNS | Sensor tidak terpasang, exclusion, sampling, field tidak dikumpulkan.    |
| Network     | Flow, proxy, DNS, firewall, pcap     | Encrypted payload, east-west blind spot, NAT/proxy mengubah attribution. |
| Identity    | Logon, token, MFA, SSO, directory    | Shared account, missing device context, token/session reuse.             |
| Application | Web/API/database/business audit      | Logging tidak konsisten atau hanya error-level.                          |
| Cloud/SaaS  | Control-plane/API/identity audit     | Retention pendek, region/account tidak ter-onboard.                      |

Blind spot harus ditulis sebagai limitation, bukan ditutupi dengan asumsi. Detection-aware operator mendesain test sesuai telemetry yang tersedia atau secara eksplisit mengklasifikasikan langkah sebagai not-testable.

3. ATT&CK Detection Strategies: Model Defensive yang Lebih Baru

ATT&CK v19.2 adalah current content version. Sejak ATT&CK v18, “detections” pada technique diganti oleh Detection Strategies dan Analytics; Data Sources juga deprecated untuk penambahan baru. Detection Strategy adalah pendekatan tingkat tinggi untuk mendeteksi behavior, sedangkan Analytics mendeskripsikan logic/platform-specific detection. Ini cocok untuk Red Team: jangan bertanya hanya “event apa yang keluar?”, tetapi “kombinasi telemetry + analytic apa yang harus membedakan behavior dari baseline?”.

3.1 Dari behavior ke detection hypothesis

Behavior -> Expected telemetry -> Analytic idea -> Context -> Triage -> Outcome

| Tahap              | Contoh discovery test                                                                |
|--------------------|--------------------------------------------------------------------------------------|
| Behavior           | User menjalankan read-only system discovery di test VM.                              |
| Expected telemetry | Process creation + command line + user + parent process.                             |
| Analytic idea      | Rare admin/discovery utility chain pada endpoint yang biasanya tidak menjalankannya. |
| Context            | Host role, operator allow-list, maintenance window, signer/path.                     |
| Triage             | Apakah behavior expected, authorized, dan sesuai change window?                      |
| Outcome            | blocked / detected / logged-only / missed / not-testable.                            |

3.2 Stealth vs Defense Impairment

ATT&CK v19 memisahkan tactic lama Defense Evasion menjadi Stealth dan Defense Impairment. Untuk module ini, keduanya dipelajari sebagai threat-model/detection domain: apa telemetry yang membantu mendeteksi upaya menyembunyikan behavior atau melemahkan pertahanan. Kita tidak mempraktikkan teknik untuk menonaktifkan sensor, menghapus log, atau membypass endpoint controls.

4. Detection Engineering untuk Red Teamers

Red Team tidak harus menjadi SIEM engineer, tetapi perlu memahami unit dasar detection: log source, fields, selection, condition, context, false positives, severity, dan correlation. Tanpa itu, hasil emulation sering berhenti di “EDR alert / tidak alert” tanpa menjelaskan mengapa.

4.1 Rule tunggal vs correlation

Rule tunggal cocok untuk signal yang jelas. Correlation menggabungkan beberapa event dalam waktu/identity/host yang sama. Sigma menyediakan generic rule format; specification 2.1.0 mendefinisikan sections seperti title, logsource, detection, condition, fields, falsepositives, level, dan tags. Sigma juga memiliki correlation rule specification untuk menggabungkan beberapa rule/event pattern. Dalam module ini kita menulis rule marker sederhana dan melakukan correlation dengan synthetic events agar aman.

4.2 Detection quality

| Dimensi       | Pertanyaan                                                                          |
|---------------|-------------------------------------------------------------------------------------|
| Fidelity      | Apakah signal benar-benar terkait behavior yang ingin dideteksi?                    |
| Coverage      | Platform/host mana yang punya telemetry? Apa gap-nya?                               |
| Context       | Apakah analytic memakai role host, identity, signer, path, atau maintenance window? |
| Timeliness    | Seberapa cepat signal tersedia dan ditriage?                                        |
| Durability    | Apakah rule bergantung pada IOC rapuh atau behavior yang lebih stabil?              |
| Actionability | Apakah alert memberi data cukup untuk analyst mengambil keputusan?                  |

4.3 False positive dan false negative

False positive bukan sekadar “alert salah”; sering kali analytic terlalu umum atau context kurang. False negative bisa terjadi karena telemetry tidak dikumpulkan, field hilang, analytic tidak matching, event terlambat, atau behavior berbeda dari asumsi. Sebelum menyebut “detection missed”, pastikan testable prerequisites benar-benar tersedia.

4.4 Mendesain detection test seperti eksperimen

| Elemen           | Contoh                                                          |
|------------------|-----------------------------------------------------------------|
| Baseline window  | 10 menit sebelum test pada host yang sama.                      |
| Test stimulus    | Satu behavior terdefinisi dengan marker unik dan timestamp UTC. |
| Expected signal  | Event/field/analytic yang seharusnya muncul.                    |
| Negative control | Aktivitas normal tanpa marker yang seharusnya tidak match.      |
| Confounder       | Maintenance job, updater, EDR scan, operator lain.              |
| Retest           | Stimulus sama setelah tuning detection.                         |

Gunakan satu perubahan pada satu waktu. Jika behavior, logging policy, agent version, dan detection rule berubah sekaligus, hasil retest sulit diinterpretasikan. Catat version/control state agar eksperimen reproducible.

5. Windows Telemetry: Process, Network, File, DNS

Sysmon memberi detailed endpoint telemetry ketika dikonfigurasi. Event yang berguna untuk latihan antara lain Process Create (1), Network Connect (3), File Create (11), Registry events, dan DNS Query (22). Windows Security Event 4688 juga merekam process creation; command line memerlukan policy tambahan. Tidak semua event aktif secara default, sehingga “tidak ada event” belum tentu berarti behavior tidak terlihat oleh EDR atau sensor lain.

5.1 Read-only review commands

Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 50  
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4688; StartTime=(Get-Date).AddHours(-1)} -MaxEvents 50  
Script lab windows-telemetry.ps1 hanya membaca event yang sudah ada. Ia tidak mengaktifkan auditing, tidak mengubah Sysmon config, dan tidak membersihkan event log. Jika log/channel tidak tersedia, hasil dicatat sebagai telemetry gap.

5.2 Windows event map untuk latihan

| Event / source | Signal             | Catatan                                                                            |
|----------------|--------------------|------------------------------------------------------------------------------------|
| Sysmon 1       | Process creation   | Image, parent, command line, hashes tergantung config.                             |
| Sysmon 3       | Network connection | Berguna untuk process-to-destination correlation; disabled by default pada Sysmon. |
| Sysmon 11      | File create        | Hubungkan file path dengan process dan timing.                                     |
| Sysmon 22      | DNS query          | Process-aware DNS visibility jika dikonfigurasi.                                   |
| Security 4688  | Process creation   | CommandLine perlu policy tambahan; field bisa kosong.                              |
| Security 4624  | Successful logon   | Penting untuk identity/source/logon-type context.                                  |

Jangan menjadikan nomor event sebagai “IOC”. Event ID adalah container telemetry; deteksi membutuhkan kombinasi field, sequence, baseline, dan context.

6. Linux Telemetry: Journal, Process, Socket, Audit

6.1 Baseline read-only

ps -eo pid,ppid,user,comm,args --sort=pid | head -80  
ss -tulpn  
journalctl --since '-15 min' --no-pager | tail -200  
Structured journal dapat difilter berdasarkan unit/field. Di host lab, dokumentasikan privilege yang diperlukan untuk membaca log tertentu. Jangan mengasumsikan seluruh journal dapat dibaca sebagai unprivileged user.

6.2 Optional audit subsystem

sudo ausearch -ts recent -m EXECVE,USER\_CMD 2>/dev/null | head -80  
Jalankan hanya jika auditd/audit rules memang sudah aktif di lab. Module ini tidak memasang rule audit baru secara otomatis. Audit configuration adalah bagian dari defensive engineering dan harus dikelola terpisah dari test behavior.

6.3 Linux event anatomy

Untuk correlation, prioritaskan timestamp, host, uid/euid, pid/ppid, executable, arguments, unit/service, remote/local address, dan file path. Di systemd journal, structured fields memungkinkan filter yang lebih presisi daripada grep bebas. Di audit subsystem, satu aktivitas dapat menghasilkan beberapa record yang perlu digabung berdasarkan event serial/timestamp.

6.4 Time synchronization

Korelasi process, network, SIEM, dan operator log runtuh jika clock berbeda jauh. Gunakan UTC pada evidence dan catat offset bila sistem tidak sinkron. Jangan “memperbaiki” waktu production saat test; time drift adalah finding/limitation yang harus dilaporkan melalui owner yang tepat.

7. Safe Detection Lab: Synthetic + Benign Real Activity

Kita memisahkan dua jenis evidence: (1) synthetic event stream untuk belajar rule/correlation secara deterministik, dan (2) benign real activity di localhost untuk melihat process/file/network footprint tanpa payload ofensif. Dengan pemisahan ini, hasil mudah direproduksi dan tidak tergantung produk EDR tertentu.

7.1 Workspace

cd modules/24-opsec-detection-aware-operations/labs  
./workspace-init.sh module24-opsec

7.2 OPSEC plan validation

python opsec-plan.py validate sample-opsec-plan.json  
python opsec-plan.py render sample-opsec-plan.json --output module24-opsec/admin/opsec-plan.md  
Validator memeriksa objective, scope, allowed/prohibited actions, stop conditions, exposure budget, telemetry expectations, cleanup owner, dan evidence plan. PASS hanya berarti struktur lengkap; bukan pengganti authorization.

7.3 Benign activity simulation

python benign-activity.py --workspace module24-opsec --marker RTLAB24\_MARKER  
Simulator melakukan read-only identity/system discovery, membuat temporary marker file di workspace, membuka local loopback HTTP service sementara, melakukan satu request ke localhost, menghitung SHA-256, lalu menghapus marker. Tidak ada remote target, persistence, privilege escalation, atau security-control modification.

7.4 Synthetic telemetry

python synthetic-events.py --output module24-opsec/telemetry/events.jsonl  
python correlate-events.py module24-opsec/telemetry/events.jsonl

| Synthetic event | Tujuan belajar                                    |
|-----------------|---------------------------------------------------|
| process_create  | Process image, parent, user, command line marker. |
| file_create     | Korelasi file path/hash dengan process.           |

|                 |                                                   |
|-----------------|---------------------------------------------------|
| dns_query       | Memahami identity process + DNS context.          |
| network_connect | Destination/port/timestamp dan sequence.          |
| cleanup         | Membedakan test completion dengan residual state. |

## 7.5 Evidence normalization

Simpan raw evidence apa adanya, lalu buat normalized copy untuk analisis. Jangan mengedit raw log agar “lebih rapi”. Minimum normalized fields: timestamp\_utc, host, user, process, parent, event\_type, destination/path, marker/test\_id, dan source. Raw evidence + hash memungkinkan reviewer mengulang transformasi bila diperlukan.

## 7.6 Marker strategy

Marker seperti RTLAB24\_MARKER membantu attribution di lab, tetapi jangan menganggap marker sebagai detection strategy. Production analytic harus mendeteksi behavior/context yang relevan. Marker hanya alat eksperimen untuk memastikan stimulus, event, dan operator activity dapat dihubungkan secara deterministic.

# 8. Sigma Fundamentals dengan Marker Aman

## 8.1 Training rule

```
title: Red Team Lab Marker In Process Command Line
status: experimental
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    CommandLine|contains: 'RTLAB24_MARKER'
  condition: selection
falsepositives:
  - Authorized Module 24 training
level: low
```

Rule ini sengaja mendeteksi marker yang kita kontrol sendiri. Ia bukan production rule. Tujuannya memahami struktur Sigma, false-positive declaration, tagging, dan bagaimana detection logic harus diuji terhadap data positif maupun negatif.

## 8.2 Structural check

```
python sigma-structure.py sigma/training-process-marker.yml
```

Helper hanya memeriksa struktur minimum untuk latihan; bukan pengganti official schema validator/pySigma. Di production pipeline, gunakan tooling Sigma yang sesuai backend dan version pinning.

## 8.3 Rule lifecycle

Hypothesis -> Sample data -> Rule draft -> Positive test -> Negative test -> Tuning -> Peer review -> Deploy -> Monitor -> Retest

Rule yang “match test” belum tentu production-ready. Dokumentasikan field dependencies, false positives yang diketahui, backend assumptions, dan data-retention requirements. Ketika rule dikonversi ke backend SIEM tertentu, verifikasi semantic equivalence; correlation feature dapat berbeda antar-backend.

# 9. Detection Evaluation: Dari “Alert Ada” ke Hasil yang Bisa Dipertanggungjawabkan

| Status       | Definisi                                                                       |
|--------------|--------------------------------------------------------------------------------|
| blocked      | Control mencegah behavior yang direncanakan.                                   |
| detected     | Telemetry + analytic menghasilkan signal actionable.                           |
| logged-only  | Evidence tersedia tetapi analytic tidak menghasilkan detection.                |
| missed       | Telemetry yang diperlukan tersedia, behavior terjadi, analytic expected gagal. |
| not-testable | Precondition/telemetry/scope tidak cukup untuk menarik kesimpulan.             |

## 9.1 Expected-vs-observed matrix

```
python detection-evaluate.py sample-observations.json --output module24-opsec/results/evaluation.md
```

Evaluator memaksa setiap step memiliki expected telemetry, observed evidence, analytic expectation, dan classification. Dengan begitu, “tidak ada alert” tidak otomatis dicap missed.

## 9.2 Evidence confidence

| Confidence | Kapan digunakan                                                                                    |
|------------|----------------------------------------------------------------------------------------------------|
| High       | Raw event tersedia, field sesuai, timestamp/host identity jelas, analytic behavior terverifikasi.  |
| Medium     | Sebagian evidence tersedia tetapi ada enrichment/field gap yang tidak mematahkan kesimpulan utama. |
| Low        | Kesimpulan bergantung pada inference, indirect evidence, atau timestamp/context yang lemah.        |

## 9.3 Jangan campur prevention dengan detection

Jika control memblokir behavior sebelum event downstream terjadi, absence dari event downstream bukan otomatis telemetry gap. Catat prevention evidence secara terpisah dan jelaskan bagian chain mana yang tidak sempat berjalan. Detection evaluation harus membedakan “tidak terjadi”, “terjadi tetapi tidak terkumpul”, dan “terkumpul tetapi tidak terdeteksi”.

## 9.4 Telemetry gap register

| Gap            | Contoh                    | Action                                                 |
|----------------|---------------------------|--------------------------------------------------------|
| Collection gap | Sysmon EID 3 tidak aktif. | Catat not-testable; jangan menebak network visibility. |

|              |                                            |                                                               |
|--------------|--------------------------------------------|---------------------------------------------------------------|
| Field gap    | 4688 ada, CommandLine kosong.              | Review policy / alternate EDR telemetry.                      |
| Coverage gap | Linux host tidak memiliki audit rules.     | Gunakan journal/process telemetry atau plan defensive change. |
| Context gap  | Rule match tapi host role tidak diketahui. | Enrich asset/identity context.                                |

## 10. Atomic Red Team: Review, Execute, Observe, Cleanup

Atomic Red Team adalah library test kecil yang dipetakan ke ATT&CK. Dokumentasi resminya menekankan permission, test machine, memahami dependencies/commands, dan cleanup. Module ini memakai review-first workflow. Eksekusi optional hanya untuk low-risk discovery test pada disposable VM dengan EDR/logging aktif.

Invoke-AtomicTest <TECHNIQUE> -ShowDetailsBrief

Invoke-AtomicTest <TECHNIQUE> -ShowDetails

Sebelum execute: baca attack commands, dependencies, input defaults, cleanup commands, expected artifacts, dan scope. Jangan otomatis install prerequisites dari internet pada workstation sensitif tanpa review provenance.

### 10.1 Selection gate

| Gate          | Pertanyaan                                                               |
|---------------|--------------------------------------------------------------------------|
| Authorization | Apakah host/technique/impact benar-benar diizinkan?                      |
| Reversibility | Apakah cleanup diketahui dan dapat diverifikasi?                         |
| Dependencies  | Apakah binary/script berasal dari sumber terpercaya dan version pinned?  |
| Privilege     | Apakah test membutuhkan elevation yang tidak diperlukan untuk objective? |
| Blast radius  | Apakah test menyentuh network/shared resource/identity di luar VM?       |
| Telemetry     | Apakah sensor yang ingin diuji aktif sebelum execution?                  |

### 10.2 Retest discipline

Baseline -> Atomic/benign test -> Observe -> Tune detection -> Retest -> Compare

Retest harus menggunakan precondition dan test variant yang sama sejauh mungkin agar perbandingan valid. Perubahan environment, agent version, policy, atau input harus dicatat.

## 11. Operator Hygiene, Provenance, dan Change Control

Detection-aware operation juga membutuhkan hygiene yang bisa diaudit. Setiap operator sebaiknya menggunakan identity individual, bukan shared account; tool/artifact harus punya provenance dan hash; system clock harus sinkron; dan setiap perubahan yang disengaja harus dapat ditautkan ke ticket/engagement step. Kebiasaan ini membantu incident responder membedakan test authorized dari aktivitas tak dikenal tanpa mengandalkan tebakan.

### 11.1 Secrets dan command history

| Risiko                          | Praktik yang lebih aman                                                                                   |
|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| Password/token di CLI           | Gunakan prompt interaktif, stdin, scoped secret store, atau file berpermission ketat jika tool mendukung. |
| Shared operator credential      | Identity per operator; audit trail harus bisa menunjuk siapa mengirim task.                               |
| Random binary/tool              | Ambil dari upstream resmi, pin version, verifikasi signature/hash, simpan provenance.                     |
| Timestamp lokal tidak konsisten | Catat UTC dan pastikan clock sinkron agar host/network/SIEM evidence dapat dikorelasikan.                 |
| Artifact tanpa owner            | Set owner + purpose + created_at + cleanup state pada staging inventory.                                  |

### 11.2 Activity log minimum

timestamp\_utc | operator | host | action | tool\_version | expected\_footprint | result | cleanup\_state

Activity log bukan pengganti raw telemetry. Fungsinya adalah konteks engagement: apa yang sengaja dilakukan, kapan, oleh siapa, terhadap aset mana, dan kapan dibersihkan. Simpan log ini terpisah dari evidence produk keamanan agar tidak terjadi circular validation.

### 11.3 Change control

Jika test memerlukan perubahan konfigurasi - misalnya mengaktifkan logging tambahan pada lab - pisahkan perubahan defensive instrumentation dari behavior under test. Catat before/after state dan siapa yang menyetujui. Jangan mengubah security policy di production hanya untuk “membuat lab bekerja” tanpa change process.

## 12. Cleanup & Residual Verification

Cleanup bukan sekadar delete file. Verifikasi listener berhenti, process training berakhir, temporary artifact hilang, test account/token tidak tersisa, scheduled execution tidak dibuat, firewall/config tidak berubah, container/network dibersihkan, dan evidence yang perlu dipertahankan dipindahkan ke storage yang benar.

### 12.1 Checklist

| Area       | Verifikasi                                                             |
|------------|------------------------------------------------------------------------|
| Process    | Tidak ada training process/listener yang masih berjalan.               |
| Network    | Tidak ada port/listener lab yang tertinggal.                           |
| Filesystem | Temporary marker/artifact sudah dihapus; evidence final tetap ada.     |
| Identity   | Tidak ada temporary account/token/credential yang tersisa.             |
| Config     | Tidak ada policy, service, task, registry/config change tak terencana. |
| Evidence   | Manifest hash dibuat setelah cleanup documentation selesai.            |

## 13. Guided Labs

| Lab                            | Outcome                                                        |
|--------------------------------|----------------------------------------------------------------|
| 01 - Workspace & OPSEC charter | Buat folder, plan, scope, stop condition, cleanup owner.       |
| 02 - Footprint inventory       | Prediksi process/network/file/DNS/auth footprint sebelum test. |

|                               |                                                                 |
|-------------------------------|-----------------------------------------------------------------|
| 03 - Benign Omarchy activity  | Jalankan simulator localhost dan kumpulkan evidence.            |
| 04 - Synthetic events         | Generate deterministic process/file/DNS/network JSONL.          |
| 05 - Correlation              | Hubungkan events berdasarkan host/user/marker/time.             |
| 06 - Windows telemetry        | Review Sysmon + 4688 read-only; catat channel gap.              |
| 07 - Linux telemetry          | Review ps/ss/journal; optional ausearch jika sudah tersedia.    |
| 08 - Sigma rule               | Baca + validate marker rule; buat negative sample.              |
| 09 - Detection quality        | Nilai fidelity, context, false positives, actionability.        |
| 10 - Evaluation matrix        | Classify blocked/detected/logged-only/missed/not-testable.      |
| 11 - Telemetry gap register   | Pisahkan collection, field, coverage, context gaps.             |
| 12 - Atomic review-only       | Show details dan audit commands/dependencies/cleanup.           |
| 13 - Optional low-risk Atomic | Execute satu discovery test di disposable VM.                   |
| 14 - Cleanup verification     | Pastikan process/listener/artifacts sudah kembali baseline.     |
| 15 - After Action Review      | Tulis apa yang terlihat, tidak terlihat, dan perubahan deteksi. |

## 14. Capstone - Detection-Aware Discovery Exercise

Objective: rancang dan jalankan satu discovery chain benign pada authorized lab, dengan OPSEC plan, expected telemetry, evidence, detection hypothesis, cleanup proof, dan retest. Jangan menggunakan bypass/impairment techniques. Yang dinilai adalah kualitas reasoning dan evidence.

### PLAN

scope + exposure budget + stop conditions

### PREDICT

process + file + network + DNS + auth telemetry

### EXECUTE

benign discovery / localhost simulation

### OBSERVE

endpoint + network + logs

### EVALUATE

blocked / detected / logged-only / missed / not-testable

### CLEAN

verify residual state

### REPORT

evidence + gaps + remediation + retest

### Deliverables capstone

- admin/opsec-plan.md dan scope.md.
- notes/footprint-prediction.md.
- telemetry/ raw/synthetic evidence dan relevant OS log excerpts.
- detections/ Sigma marker rule + detection-hypothesis.md.
- results/evaluation.md + telemetry-gaps.md.
- cleanup/cleanup-proof.md + evidence-manifest.sha256.

### 14.1 Rubric capstone

| Area                | Bobot | Kriteria                                                                 |
|---------------------|-------|--------------------------------------------------------------------------|
| Safety & scope      | 20%   | Tidak ada out-of-scope traffic/control impairment; stop condition jelas. |
| Prediction          | 15%   | Footprint dan expected telemetry ditulis sebelum execution.              |
| Evidence            | 20%   | Raw evidence, UTC timestamp, hashes, dan provenance tersedia.            |
| Detection reasoning | 20%   | Classification didukung telemetry prerequisite + analytic evidence.      |
| Cleanup             | 15%   | Residual state diverifikasi, bukan sekadar diasumsikan.                  |
| Communication       | 10%   | Summary singkat, limitation, remediation, dan retest plan jelas.         |

## 15. Troubleshooting

| Symptom                           | Kemungkinan                               | Langkah aman                                                      |
|-----------------------------------|-------------------------------------------|-------------------------------------------------------------------|
| 4688 ada tapi command line kosong | Policy include command line tidak aktif.  | Catat field gap; jangan mengubah policy tanpa approval.           |
| Sysmon log tidak ada              | Sysmon tidak terpasang/channel disabled.  | Classify not-testable untuk Sysmon-specific expectation.          |
| journalctl minim event            | Permission/retention/unit filtering.      | Cek user access dan filter; jangan hapus/rotate log.              |
| Sigma rule tidak match            | Field taxonomy tidak sama dengan sample.  | Normalisasi field mapping; pertahankan test marker.               |
| Tidak ada alert                   | Bisa analytic gap atau telemetry gap.     | Buktikan collection + field availability sebelum menyebut missed. |
| Cleanup check gagal               | Residual process/file/listener masih ada. | Stop/remove hanya artifact milik lab, lalu verifikasi ulang.      |

## 16. Knowledge Check

- Kenapa OPSEC engagement bukan sinonim dari “menghindari detection”?
- Apa perbedaan footprint prediction dan telemetry observation?
- Kenapa command-line logging dapat sekaligus membantu detection dan membocorkan secret?
- Apa perubahan ATT&CK v18 terkait Detection Strategies dan Data Sources?
- Apa beda blocked, detected, logged-only, missed, dan not-testable?

- Kapan “tidak ada alert” boleh diklasifikasikan missed?
- Apa beda single-event rule dengan correlation?
- Kenapa Sigma rule lab memakai marker yang kita kontrol?
- Apa yang harus direview sebelum menjalankan Atomic test?
- Apa saja yang wajib diverifikasi saat cleanup?

17. Cheat Sheet

```
# Windows - read-only
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 50
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4688; StartTime=(Get-Date).AddHours(-1)} -MaxEvents 50

# Linux - read-only
ps -eo pid,ppid,user,comm,args --sort=pid | head -80
ss -tulpn
journalctl --since '-15 min' --no-pager | tail -200

# Module lab
./workspace-init.sh module24-opsec
python opsec-plan.py validate sample-opsec-plan.json
python benign-activity.py --workspace module24-opsec --marker RTLAB24_MARKER
python synthetic-events.py --output module24-opsec/telemetry/events.jsonl
python correlate-events.py module24-opsec/telemetry/events.jsonl
python sigma-structure.py sigma/training-process-marker.yml
python detection-evaluate.py sample-observations.json --output module24-opsec/results/evaluation.md
./cleanup-check.sh module24-opsec
./evidence-manifest.sh module24-opsec
```

18. Exit Criteria

| Kompetensi          | Lulus jika...                                                                               |
|---------------------|---------------------------------------------------------------------------------------------|
| OPSEC planning      | Bisa mendefinisikan exposure budget, stop condition, cleanup owner, dan prohibited actions. |
| Footprint reasoning | Bisa memprediksi process/network/file/DNS/auth footprint sebelum test.                      |
| Telemetry literacy  | Bisa membaca dasar Sysmon/4688 dan Linux journal/process/socket evidence.                   |
| Detection reasoning | Bisa membedakan telemetry, analytic, context, alert, dan outcome.                           |
| Sigma literacy      | Bisa membaca struktur rule sederhana dan menjelaskan false positives.                       |
| Evaluation          | Tidak menyebut missed sebelum membuktikan telemetry prerequisite tersedia.                  |
| Cleanup             | Mampu membuktikan residual lab sudah dibersihkan dan evidence di-hash.                      |

19. Referensi Utama

| Sumber                                                | URL                                                                                                                                                                                                                                             |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MITRE ATT&CK v19.2 - Version History                  | <a href="https://attack.mitre.org/resources/versions/">https://attack.mitre.org/resources/versions/</a>                                                                                                                                         |
| MITRE ATT&CK - Detection Strategies                   | <a href="https://attack.mitre.org/detectionstrategies/">https://attack.mitre.org/detectionstrategies/</a>                                                                                                                                       |
| MITRE ATT&CK v18 defensive model changes              | <a href="https://attack.mitre.org/resources/updates/updates-october-2025/">https://attack.mitre.org/resources/updates/updates-october-2025/</a>                                                                                                 |
| MITRE ATT&CK v19 - Stealth / Defense Impairment split | <a href="https://attack.mitre.org/resources/updates/updates-april-2026/">https://attack.mitre.org/resources/updates/updates-april-2026/</a>                                                                                                     |
| Microsoft - Sysmon events                             | <a href="https://learn.microsoft.com/windows/security/operating-system-security/sysmon/sysmon-events">https://learn.microsoft.com/windows/security/operating-system-security/sysmon/sysmon-events</a>                                           |
| Microsoft - Event 4688 Process Creation               | <a href="https://learn.microsoft.com/windows/security/threat-protection/auditing/event-4688">https://learn.microsoft.com/windows/security/threat-protection/auditing/event-4688</a>                                                             |
| Microsoft - Command line process auditing             | <a href="https://learn.microsoft.com/windows-server/identity/ad-ds/manage/component-updates/command-line-process-auditing">https://learn.microsoft.com/windows-server/identity/ad-ds/manage/component-updates/command-line-process-auditing</a> |
| systemd - journalctl                                  | <a href="https://www.freedesktop.org/software/systemd/man/latest/journalctl.html">https://www.freedesktop.org/software/systemd/man/latest/journalctl.html</a>                                                                                   |
| Linux auditd manual                                   | <a href="https://man7.org/linux/man-pages/man8/auditd.8.html">https://man7.org/linux/man-pages/man8/auditd.8.html</a>                                                                                                                           |
| Sigma Specification 2.1.0                             | <a href="https://github.com/SigmaHQ/sigma-specification">https://github.com/SigmaHQ/sigma-specification</a>                                                                                                                                     |
| Atomic Red Team - Getting Started                     | <a href="https://www.atomicredteam.io/docs/atomic-red-team">https://www.atomicredteam.io/docs/atomic-red-team</a>                                                                                                                               |

Catatan versi: materi defensif ATT&CK pada modul ini mengikuti model v19.2 yang current saat modul disusun. Karena ATT&CK, Sysmon, Sigma, dan Atomic Red Team terus berubah, review referensi resmi sebelum menjalankan lab atau menerapkan detection di production.